

Product Requirements Document

A Fully Local Multilingual Retrieval-Augmented Generation System with Semantic Chunking, Hybrid Retrieval, and Neural Reranking for Document Question Answering

Field	Detail
Short Name	Local Multilingual RAG
Course	CSE-460 (Deep Learning)
Document Type	Product Requirements Document (PRD)
Status	Locked — Planning Complete, Pre-Implementation
Version	1.0

1. Executive Summary

This document specifies the requirements for a **fully local, multilingual Retrieval-Augmented Generation (RAG) system** that allows users to upload research papers and other documents, ask questions in **English or Bangla**, and receive answers grounded in the uploaded documents with page-level and section-level citations. The system uses no external AI APIs or API keys — all embedding, retrieval, reranking, and generation components run locally, and the system continues to function offline once models are downloaded.

Beyond functioning as a document question-answering tool, the project has a genuine research component: it systematically investigates how **chunking strategy** (fixed vs. semantic), **retrieval strategy** (BM25 vs. vector vs. hybrid), and **neural reranking** affect retrieval effectiveness and answer quality, across both English and Bangla.

2. Problem Statement & Motivation

Existing document question-answering tools are typically either (a) cloud-dependent, requiring external API keys and raising data-privacy concerns, or (b) English-only, leaving Bangla and other low-resource languages underserved. There is also limited empirical clarity, at a project scope, on how specific RAG design choices — chunk granularity, retrieval method, and reranking — actually move the needle on retrieval and answer quality, rather than being adopted purely by convention.

This project addresses both gaps: it builds a document QA system that runs entirely on local infrastructure and supports English/Bangla (including cross-lingual) querying, and it empirically measures the effect of each major design choice in the pipeline.

3. Goals and Objectives

- Build a working, end-to-end local RAG web application for document upload and question answering.
- Support English and Bangla queries against documents in either language, including cross-lingual retrieval.
- Ground every answer in retrieved document content and surface page-level / section-level citations.
- Operate with zero external AI API dependencies — all inference is local and works offline once set up.
- Produce a rigorous empirical comparison of chunking, retrieval, and reranking strategies to answer the project's research questions.

4. Scope

4.1 In Scope — Version 1

- Document upload and processing for PDF, DOCX, TXT, and Markdown.
- OCR fallback for scanned/image-based documents.
- Fixed-size and semantic/structure-aware chunking, implemented and compared.
- Local multilingual embeddings (BGE-M3) and local vector storage (Qdrant).
- BM25 keyword retrieval, vector retrieval, and hybrid (fused) retrieval.
- Neural reranking of hybrid retrieval results (BGE-reranker-v2-m3).
- Local LLM answer generation (Qwen, with Llama/Mistral as fallbacks) via Ollama.
- Citation system tying every answer back to source document, page, and section.
- “Not found” handling when the answer is not supported by retrieved context.
- English, Bangla, and cross-lingual (Bangla question → English document) query support.
- A structured evaluation dataset and the full experiment matrix described in Section 10.

4.2 Optional — Version 1 (Time Permitting)

- Support for PPTX, CSV, and HTML document types.
- Research-paper structural parsing (title, authors, abstract, keywords, sections, references) surfaced in the UI.

4.3 Out of Scope — Deferred to Version 2

- User authentication and multi-user accounts.
- Persistent chat history across sessions.
- Document management (organizing, deleting, tagging documents).
- Paper-to-paper comparison feature.
- Redis, Celery, Docker-based deployment, admin dashboard, and production monitoring (OpenTelemetry, Langfuse).

4.4 Explicit Non-Goals

- The project will not train a large language model from scratch — pretrained Transformer models are used for embeddings, reranking, and generation.
- The system will not use any external AI API (OpenAI, Gemini, Claude, Cohere, Pinecone Cloud, etc.).
- The project will not claim to eliminate hallucination or to support millions of documents; claims are scoped to what is actually benchmarked.

5. Target Users

- **Researchers / students** who need to query long research papers in English or Bangla and get grounded, cited answers.
- **Course evaluators / examiners** assessing the project's research contribution around chunking, retrieval, and reranking.
- **The developer (project author)**, who also uses the system as an experimental testbed for the RAG ablation study.

6. Research Contribution & Research Questions

The project is explicitly framed as more than a “PDF chatbot.” Its core research contribution is an empirical study of how chunking and retrieval design choices affect multilingual RAG performance.

6.1 Main Research Question

How do semantic chunking, hybrid retrieval, and neural reranking affect the retrieval effectiveness, answer quality, and faithfulness of a fully local multilingual RAG system for document question answering?

6.2 Supporting Research Questions

- **RQ1:** Does semantic chunking improve retrieval compared with fixed-size chunking?
- **RQ2:** Does hybrid BM25 + vector retrieval outperform individual retrieval methods?
- **RQ3:** Does neural reranking improve the relevance of retrieved context?
- **RQ4:** How does the system perform for English and Bangla queries?
- **RQ5:** Can a fully local RAG system provide accurate and grounded answers without external AI APIs?

6.3 Hypothesis

Semantic chunking, hybrid retrieval, and neural reranking are *expected* to improve retrieval quality and downstream answer quality compared with simpler RAG configurations — but this is a hypothesis to be tested experimentally, not an assumed conclusion.

7. Functional Requirements

ID	Requirement	Description
FR-1	Document Upload	User can upload PDF, DOCX, TXT, or Markdown files through the web UI.
FR-2	File Validation	System validates file type/size and rejects unsupported formats gracefully.
FR-3	Text Extraction	System extracts text and structure from documents using Docling / PyMuPDF / python-docx.
FR-4	OCR Fallback	System applies OCR (Tesseract, or PaddleOCR if it benchmarks better) to scanned/image-based content.
FR-5	Chunking	System supports both fixed-size chunking and semantic/structure-aware chunking, selectable for experimentation.
FR-6	Metadata Tagging	Each chunk is tagged with document ID, document name, page number, section, and chunk ID.

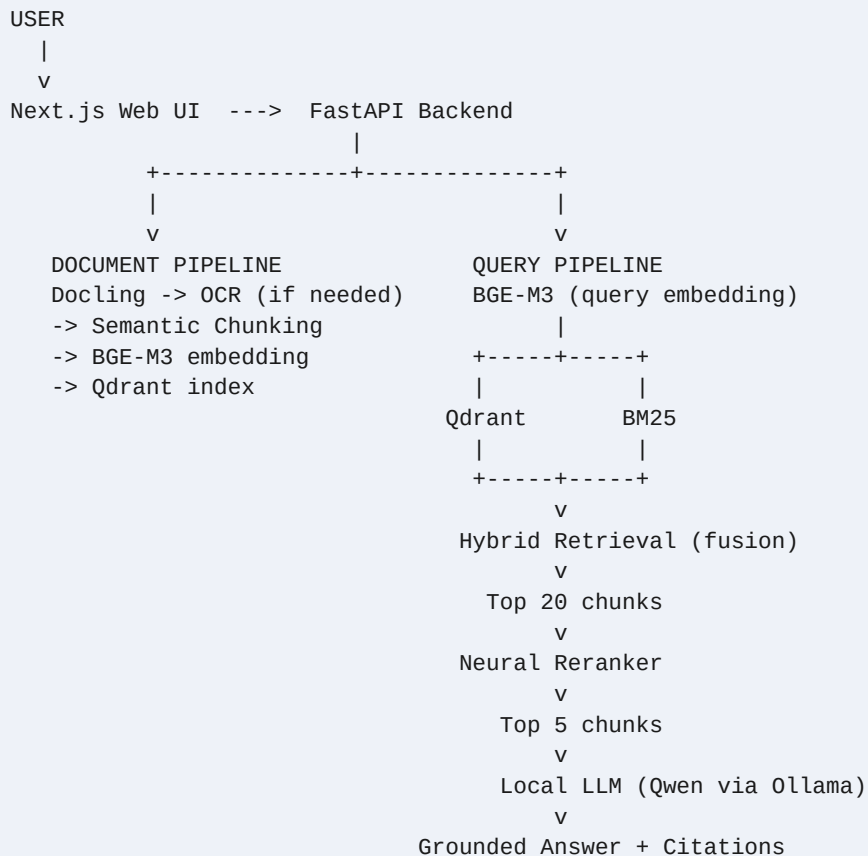
ID	Requirement	Description
FR-7	Embedding Generation	System generates multilingual embeddings for each chunk using BGE-M3 and stores them in Qdrant.
FR-8	Keyword Retrieval	System supports BM25 keyword-based retrieval over indexed chunks.
FR-9	Vector Retrieval	System supports dense vector retrieval over Qdrant using BGE-M3 query embeddings.
FR-10	Hybrid Retrieval	System fuses BM25 and vector search results into a single ranked list (top 20).
FR-11	Neural Reranking	System reranks the top 20 hybrid results with BGE-reranker-v2-m3 down to a top 5 context set.
FR-12	Answer Generation	System sends the question and top-5 context to a local LLM (via Ollama) to produce a grounded answer.
FR-13	Citation Display	Every answer is accompanied by source document name, page number, and section, derived from chunk metadata.
FR-14	Not-Found Handling	If retrieved context does not support an answer, the system responds that the information was not found, rather than fabricating an answer.
FR-15	Multilingual Querying	System accepts English and Bangla questions and supports English→English, Bangla→Bangla, and Bangla→English (cross-lingual) retrieval.
FR-16	Research Paper Structuring (optional)	System extracts and displays title, authors, abstract, keywords, and section outline for research-paper uploads.
FR-17	Experiment Instrumentation	System can run and log the defined experiment matrix (chunking, retrieval, reranking, language) for evaluation.

8. Non-Functional Requirements

ID	Requirement	Description
NFR-1	Locality	No external AI API calls of any kind; all inference (embedding, reranking, generation) runs on local infrastructure.
NFR-2	Offline Operation	Once models and packages are downloaded, the system must function without an internet connection.
NFR-3	Groundedness	Generated answers must be grounded in retrieved context; the system must not present fabricated claims as fact.
NFR-4	Traceability	Every answer must be traceable to specific source chunks via stored metadata.
NFR-5	Multilingual Robustness	Retrieval and generation quality should be evaluated, not assumed, for both English and Bangla.
NFR-6	Performance Visibility	Retrieval, reranking, generation, and total query latency, plus memory usage, must be measurable.
NFR-7	Modularity	Pipeline stages (chunking, retrieval, reranking, generation) must be swappable for experimentation.
NFR-8	Honest Claims	Documentation and presentation must scope claims to what has actually been benchmarked (see Section 14).

9. System Architecture

At a high level, the system splits into a document pipeline and a query pipeline, converging at hybrid retrieval:



Document ingestion flow: Upload → File Validation → Document Type Detection → Text Extraction → OCR (if necessary) → Structure Detection → Chunking → Metadata Generation → Embedding → Qdrant indexing.

9.1 Chunk Metadata Schema

```
{  
  "document_id": "123",  
  "document_name": "paper.pdf",  
  "page_number": 7,  
  "section": "Methodology",  
  "chunk_id": "123_007_04"  
}
```

10. Technology Stack

Layer	Choice
Frontend	Next.js, TypeScript, Tailwind CSS, shadcn/ui
Backend	Python, FastAPI
Document Processing	Docling, PyMuPDF, python-docx
OCR	Tesseract OCR (primary); PaddleOCR if it benchmarks better
Embedding Model	BAAI/bge-m3 (multilingual, incl. Bangla)
Vector Database	Qdrant (self-hosted, local)
Keyword Retrieval	BM25
Reranker	BAAI/bge-reranker-v2-m3 (local)
Local LLM	Qwen (primary candidate); Llama or Mistral as alternatives, chosen by benchmark
Local LLM Runtime	Ollama
Application Database	PostgreSQL (metadata, chat history)
Optional / Later	Redis, Celery, Docker, OpenTelemetry, Langfuse (not part of core V1)

Hard requirement: no external AI API or API key of any kind (OpenAI, Gemini, Claude, Cohere, Pinecone Cloud, etc.) may be used anywhere in the system.

11. Retrieval & Reranking Design

Three retrieval methods are implemented and compared:

Method	Name	Flow
A	BM25	Question → BM25 → Top K
B	Vector Search	Question → BGE-M3 → Qdrant → Top K
C	Hybrid Search	Question → BM25 + Vector → Result Fusion → Top 20

The exact fusion method for hybrid search is to be selected and documented during implementation.

After hybrid retrieval produces the top 20 chunks, the neural reranker (BGE-reranker-v2-m3) re-scores them for relevance and the top 5 become the context supplied to the local LLM.

12. Answer Generation, Citations, and Grounding

12.1 Local LLM Prompting

The local LLM receives system instructions, the user question, and the retrieved context. Instructions must emphasize:

- Use only the supplied context to answer.
- Do not invent information not present in the context.
- If the answer is not present in the context, state that it was not found.

12.2 Citation Format

Answer:

The researchers used a CNN-based classification approach trained on the XYZ dataset.

Sources:

[Doc] research_paper.pdf

Page 8

Section: Methodology

12.3 Not-Found Handling

If retrieved documents do not contain the answer, the system responds: “I could not find this information in the provided documents,” and must not fabricate an answer.

13. Multilingual Requirements

Initial target languages are English and Bangla. Three query/document language combinations must be tested:

- **English → English:** English question against English document.
- **Bangla → Bangla:** Bangla question against Bangla document.
- **Cross-lingual:** Bangla question against English document, where supported by the model and dataset.

The cross-lingual case is expected to be one of the project's strongest demonstrations.

14. Evaluation Plan

14.1 Evaluation Dataset

A dedicated question-answer evaluation dataset must be created, with enough examples to make results meaningful (not just five or ten questions). Each item specifies the source document, question, expected answer, and relevant page. Question categories to cover:

- Factual questions
- Methodology questions
- Dataset questions
- Results questions
- Comparison questions
- Bangla questions
- English questions

14.2 Experiment Matrix

Experiment	Comparison
1	Fixed Chunking vs. Semantic Chunking
2	BM25 vs. Vector Search
3	Vector Search vs. Hybrid Search
4	Hybrid Search vs. Hybrid + Reranker
5	English vs. Bangla
6 (optional)	Local LLM size/model comparison (quality, faithfulness, latency, memory)

14.3 Metrics

Category	Metrics
Retrieval	Recall@5, Recall@10, Recall@20, Precision@K, MRR, NDCG
Generation	Answer Relevance, Faithfulness, Context Relevance, Citation Accuracy
System	Retrieval Latency, Reranking Latency, Generation Latency, Total Query Latency, Memory Usage

14.4 Claims Discipline

The project documentation and presentation must avoid overreaching claims. Specifically:

Avoid	Prefer
"The system eliminates hallucinations."	"The system aims to reduce unsupported generation through retrieved evidence and citation-based grounding."
"The system can handle millions of documents."	"The architecture is designed with scalable components that can support larger document collections." (unless actually benchmarked)
"Our model is trained."	"The system uses pretrained Transformer-based models for embedding, reranking, and generation."

15. Development Plan (Version 1 Phases)

Phase	Name	Description
1	Environment Setup	Python, FastAPI, Next.js, Qdrant, PostgreSQL, Ollama, BGE-M3, local LLM.
2	Document Processing	Upload → Parse → OCR → Clean → Chunk → Metadata.
3	Embeddings	BGE-M3 embedding generation → Qdrant indexing.
4	Baseline RAG	Vector Search → Context → Local LLM → Answer, working end-to-end before adding complexity.
5	BM25	Add BM25 retrieval and evaluate independently.
6	Hybrid Search	Combine BM25 + Vector Search; evaluate against baselines.
7	Reranking	Add reranker on top of hybrid search (top 20 → top 5); evaluate again.
8	Citations	Wire chunk metadata (page/section) into answer citations.
9	Multilingual Testing	Test English, Bangla, and cross-lingual querying.
10	Evaluation	Run the full experiment matrix; produce tables, graphs, metrics, and error analysis.

Guiding principle: get the baseline RAG pipeline (Phase 4) working end-to-end before layering in BM25, hybrid search, and reranking.

15.1 Version 2 (Only If Time Allows)

- Authentication and multi-user support
- Persistent chat history
- Document management UI
- Redis, Celery, Docker-based deployment
- Admin dashboard and monitoring (OpenTelemetry, Langfuse)
- Paper-to-paper comparison feature

16. Deliverables

16.1 Software

- Working web application with document upload
- Document processing pipeline including OCR
- Semantic chunking alongside fixed-size chunking
- Vector, BM25, and hybrid retrieval
- Neural reranking
- Local LLM generation
- English/Bangla querying
- Citation system

16.2 Research

- Evaluation dataset
- Baseline, retrieval, reranking, and multilingual experiments
- Quantitative metrics and error analysis

16.3 Documentation

- System architecture and database design
- RAG pipeline and model selection rationale
- Experimental methodology and results
- Limitations and future work

17. Final Presentation / Demo Script

Step	Action
1	Upload a research paper.
2	Show processing status: Extracting → Chunking → Embedding → Indexing → Completed.
3	Ask an English question, e.g. “What dataset did the authors use?”
4	Display the generated answer.
5	Display the citation: source file, page number, section.
6	Ask a Bangla question.
7	Show the Bangla / cross-lingual retrieval result.
8	Walk through the evaluation comparison: BM25 → Vector → Hybrid → Hybrid + Reranker, with measured differences.

The retrieval-method comparison in Step 8 is expected to carry more weight with examiners than UI polish.

18. Reference: Repository Structure

```

local-multilingual-rag/
├── frontend/           (Next.js app, components, lib, types)
├── backend/
│   ├── app/
│   │   ├── api/ core/ models/ schemas/
│   │   ├── ingestion/ parser.py, ocr.py, chunker.py, metadata.py
│   │   ├── rag/      embeddings.py, vector_search.py, bm25.py,
│   │   │           hybrid_search.py, reranker.py, generator.py,
│   │   │           citations.py, pipeline.py
│   │   └── main.py
│   └── tests/
├── data/               raw/ processed/ evaluation/
├── models/
├── experiments/
├── notebooks/
├── scripts/
├── docker/
├── requirements.txt
├── docker-compose.yml
└── README.md

```

19. Open Items Before Implementation

The following must be finalized before development begins:

- Hardware requirements (available RAM/VRAM, disk space for local models).
- Exact local model sizes/variants (embedding, reranker, LLM) that fit the available hardware.
- Sources for the initial document/dataset collection (research papers, English + Bangla).
- Construction of the evaluation question-answer dataset.
- Finalized local development environment setup steps.

One-sentence summary: Build and experimentally evaluate a fully local English-Bangla RAG system that uses semantic chunking, BGE-M3 embeddings, BM25 + vector hybrid retrieval, neural reranking, and a local LLM to answer questions from research and enterprise documents with verifiable citations — without using any external AI API keys.